

Servicio Nacional de Aprendizaje SENA

Centro de Gestión de Mercados, Logística y Tecnologías de la Información
Tecnólogo en Análisis y Desarrollo de Software

Fase 3. Ejecución

Evidencia GA6-220501096-AA1-EV03.

Creación de los objetos de la base de datos.

Evidencia GA6-220501096-AA1-EV04.

Elaboración de las bases de datos.

Gerson Jonnathan López Oviedo

Ficha: 3186647

Instructor:

Arnaldo Alfonso Montiel Brun

Bogotá D.C. Abril de 2026

INTRODUCCIÓN

Las bases de datos no relacionales (NoSQL) representan una evolución significativa en el almacenamiento y gestión de información, especialmente en aplicaciones que requieren flexibilidad, escalabilidad y manejo de datos semiestructurados. A diferencia de los sistemas relacionales tradicionales, MongoDB organiza la información en documentos con formato JSON, permitiendo modelar objetos del mundo real de manera natural e intuitiva.

El presente trabajo unifica las evidencias GA6-220501096-AA1-EV03 y GA6-220501096-AA1-EV04, correspondientes a la creación de objetos y la elaboración de la base de datos no relacional. A lo largo del documento se aborda desde la conceptualización del modelo JSON hasta la implementación práctica en MongoDB, incluyendo la creación de la colección, inserción de documentos, consultas, actualizaciones y eliminación de registros, tomando como caso de estudio el sistema de información de un parqueadero de vehículos.

OBJETIVO GENERAL

Diseñar e implementar una base de datos no relacional en MongoDB que modele los datos de un parqueadero de vehículos, aplicando las operaciones fundamentales de creación, inserción, consulta, actualización y eliminación de documentos mediante el uso del formato JSON.

Objetivos específicos:

- Comprender la estructura y características del modelo de datos JSON aplicado a bases de datos NoSQL.
- Crear la base de datos y la colección *parque* en MongoDB siguiendo las buenas prácticas de diseño.
- Ejecutar las operaciones CRUD (Create, Read, Update, Delete) sobre la colección creada, evidenciando cada resultado.
- Aplicar filtros y consultas para recuperar documentos específicos según criterios definidos, como el número de placa del vehículo.

3. ¿Qué es el modelo JSON?

3.1 Definición

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos, de fácil lectura para humanos y sencillo de interpretar para las máquinas. Aunque nació en el ecosistema de JavaScript, es completamente independiente del lenguaje de programación y se ha convertido en el estándar de facto para el intercambio de información en APIs y bases de datos NoSQL.

3.2 Características principales

- Liviano: ocupa poco espacio comparado con XML.
- Legible: su sintaxis es intuitiva para cualquier desarrollador.
- Flexible: admite texto, números, booleanos, arreglos y objetos anidados.
- Universal: compatible con prácticamente todos los lenguajes de programación.

3.3 Estructura de un objeto JSON

Un documento JSON se compone de pares clave-valor encerrados entre llaves { }. Las claves son cadenas de texto y los valores pueden ser de distintos tipos de datos:

```
1 {  
2   "clave_texto": "valor",  
3   "clave_numero": 123,  
4   "clave_booleano": true,  
5   "clave_arreglo": [1, 2, 3],  
6   "clave_objeto": {  
7     "subclave": "subvalor"  
8   }  
9 }
```

Ejemplo:



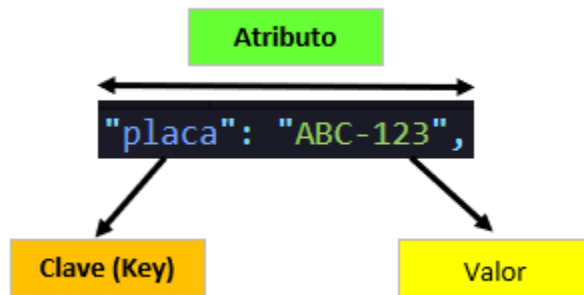
```
{ Objeto.json  
C: > Users > Sandra > Desktop > SENA OFICIAL > (1)  
1 {  
2   "clave_texto": "valor",  
3   "clave_numero": 123,  
4   "clave_booleano": true,  
5   "clave_arreglo": [1, 2, 3],  
6   "clave_objeto": {  
7     "subclave": "subvalor"  
8   }  
9 }
```

Concepto	Definición	Ejemplo en tu código
Objeto	Es todo el conjunto de datos encerrado entre llaves { }. Representa una "cosa" completa (un carro, una persona).	Todo lo que hay desde la línea 1 hasta la 10.
Clave (Key)	Es el nombre o la "etiqueta" que identifica a un dato. Siempre va a la izquierda de los dos puntos :.	"Placa", "marca", "modelo", "color"
Valor (Value)	Es la información real que contiene la clave. Va a la derecha de los dos puntos.	"Toyota", "Corolla", "Blanco".
Atributo	Es la combinación de una Clave + su Valor . Es una característica del objeto.	"color": "Blanco" es un atributo del carro.
Objeto Anidado	Es un objeto que vive dentro de otro objeto.	El campo "propietario" es un objeto con sus propias claves (nombre, cedula).

```

1  { "placa": "ABC-123",
2    "marca": "Toyota",
3    "modelo": "Corolla",
4    "color": "Blanco",
5    "propietario": {
6      "nombre": "Carlos Andrés Pérez",
7      "cedula": "1023456789",
8      "telefono": "3101234567"
9    }
10 }
```

Objeto



Objeto Anidado

```

"propietario": {
  "nombre": "Carlos Andrés Pérez",
  "cedula": "1023456789",
  "telefono": "3101234567"
}
```

3.4 Concepto de sentencia

Tener en cuenta que en el mundo de la programación y las bases de datos, una sentencia (o statement en inglés) es una instrucción completa que le das al computador para que realice una acción específica.

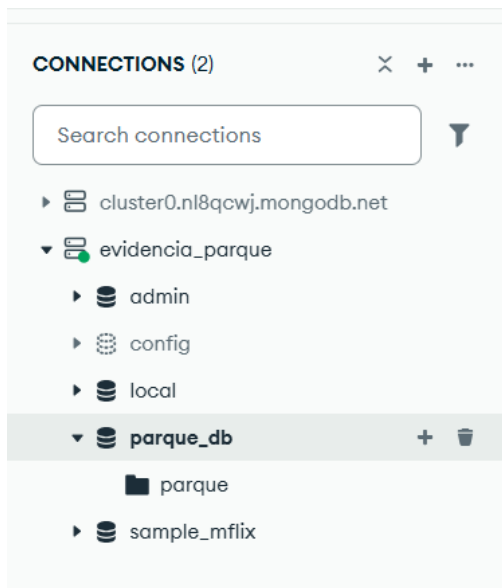
Cada paso es una instrucción individual. Si juntas muchas sentencias en orden, terminas con un algoritmo (o la receta completa). En tu código de MongoDB, cada línea que termina o que ejecuta una acción (como `db.parque`) es una sentencia que construye un proyecto:

Ejemplo:

La sentencia completa es toda la línea de código. Por ejemplo:

`db.parque.insertMany([...])`

- **db**: La base de datos.
- **parque**: La colección.
- **insertMany**: Es el método o la acción (el verbo).



3.5 ¿Cómo se usa JSON en MongoDB?

MongoDB almacena los registros como documentos BSON (Binary JSON), que es una extensión de JSON con tipos adicionales (como fechas y ObjectId). Desde el punto de vista del desarrollador, los documentos se escriben y se leen como JSON estándar. Cada documento dentro de una colección puede tener una estructura diferente, lo que otorga gran flexibilidad al modelo de datos.

4. Modelo del objeto JSON ejemplo con Vehículo

A continuación se define la estructura del documento JSON que modela los datos de un vehículo en el parqueadero. Cada campo representa un atributo del carro:

Campo	Tipo de dato	Descripción
placa	String	Número de placa del vehículo (ej. ABC-123)
numeroSerie	String	Número de serie único del vehículo
modelo	Number	Año modelo del vehículo (ej. 2022)
marca	String	Marca del fabricante (ej. Toyota)
kilometraje	Number	Kilometraje actual del vehículo
tipo	String	Tipo de vehículo: Automóvil, Moto, Campero

Sentencia para crear el objeto JSON que modela los datos de un carro:

```
Objeto.json
C: > Users > Sandra > Desktop > SENA OFICIAL > (1) Fases > (5) Fase
1
2  {
3    "placa": "ABC-123",
4    "numeroSerie": "1HGBH41JXMN109186",
5    "modelo": 2022,
6    "marca": "Toyota",
7    "kilometraje": 15000,
8    "tipo": "Automóvil"
9  }
```

5. Sentencias en MongoDB

5.1 Crear la base de datos y la colección

En MongoDB la base de datos se crea implícitamente al usarla por primera vez. El siguiente comando selecciona (o crea) la base de datos `parque_db` y crea la colección `parque`:

```
use parque_db
```

```
db.createCollection("parque")
```

5.2 Insertar 5 documentos en la colección

La sentencia `insertMany` permite insertar múltiples documentos en una sola operación:



Archivo

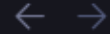
Editar

Selección

Ver

Ir

Ejecutar



EXPLORADOR

Parque.json



EDITORES ... 1 sin guardar



Parque.json C:\User...



NO HAY NINGUNA CARPE...



ESQUEMA



LÍNEA DE TIEMPO

C:\Users\Sandra\Desktop>SENA OFICIAL>(1) Fases>(5) Fase 3. Ejecución>(4)>Parque.json>{} 3

```
1  [{"Parquesadeno Central",
2    { "placa": "ABC-123",
3      "marca": "Toyota",
4      "modelo": "Corolla",
5      "color": "Blanco",
6      "propietario": {
7        "nombre": "Carlos Andrés Pérez",
8        "cedula": "1023456789",
9        "telefono": "3101234567"
10     },
11     "fecha_ingreso": "2025-04-01",
12     "hora_ingreso": "08:30",
13     "tipo_vehiculo": "Automóvil"
14   },
15   {
16     "placa": "XYZ-456",
17     "marca": "Chevrolet",
18     "modelo": "Spark",
19     "color": "Rojo",
20     "propietario": {
21       "nombre": "Laura Milena Torres",
22       "cedula": "1098765432",
23       "telefono": "3209876543"
24     },
25     "fecha_ingreso": "2025-04-01",
26     "hora_ingreso": "09:15",
27     "tipo_vehiculo": "Automóvil"
28   },
29   {}
30   { "placa": "MNO-789",
31     "marca": "Yamaha",
32     "modelo": "FZ25",
33     "color": "Negro",
34     "propietario": {
35       "nombre": "Juan Esteban Gómez",
36       "cedula": "1057894321",
37       "telefono": "3153456789"
38     },
39     "fecha_ingreso": "2025-04-02",
40     "hora_ingreso": "10:00",
41     "tipo_vehiculo": "Motocicleta"
42   },
43   {
44     "placa": "PQR-321",
45     "marca": "Renault",
46     "modelo": "Logan",
47     "color": "Gris",
48     "propietario": {
49       "nombre": "Sofía Valentina Ruiz",
50       "cedula": "1034567891",
51       "telefono": "3187654321"
52     },
53     "fecha_ingreso": "2025-04-02",
54     "hora_ingreso": "11:45",
55     "tipo_vehiculo": "Automóvil"
56   },
57   {
58     "placa": "DEF-654",
59     "marca": "Mazda",
60     "modelo": "CX-5",
61     "color": "Azul",
62     "propietario": {
63       "nombre": "Miguel Antonio Vargas",
64       "cedula": "1067891234",
65       "telefono": "3162345678"
66     },
67     "fecha_ingreso": "2025-04-03",
68     "hora_ingreso": "07:50",
69     "tipo_vehiculo": "Campero"
70   }
71 ]
72 }
```

6. Array (Arreglo) en JSON

Un Array es una estructura de datos que se utiliza para almacenar múltiples valores en una sola variable. En el formato JSON, se identifica fácilmente porque siempre va encerrado entre corchetes [].

6.1 Características clave para el proyecto:

- **Agrupación:** Permite guardar varios objetos (como tus 5 vehículos) bajo un mismo nombre de colección.
- **Orden:** Los elementos dentro de un array mantienen el orden en que fueron escritos.
- **Separación:** Cada objeto dentro del array se separa del siguiente mediante una **coma (,)**.

Ejemplo:

JSON

```
"vehiculos": [  
  { "placa": "ABC-123", "marca": "Toyota" },  
  { "placa": "XYZ-456", "marca": "Chevrolet" }  
]
```

Aquí, los corchetes le dicen a la base de datos que "vehiculos" es una lista que contiene varios elementos individuales.

La Analogía del Tren de Carga:

[Image: A train with several connected freight cars on a track]

Imagina que un **Array** es un **Tren de Carga**:

- Los **Corchetes []** son los rieles y el enganche que mantienen a todo el tren unido como una sola unidad.
- Cada **Vagón { }** es un objeto independiente (un carro con su placa y modelo).
- Las **Comas (,)** son los enganches que conectan un vagón con el siguiente.

Sin los corchetes (el tren), tendrías los vagones sueltos por toda la vía y la base de datos no sabría que todos pertenecen al mismo grupo de **TRUEQUE MATCH**.

{ } Parque.json ●

{ } Array.json ●

C: > Users > Sandra > Desktop > SENA OFICIAL > (1) Fases > (5) Fase 3. Ejecuci

```
1  {
2    "Parqueadero_Central": [
3      {
4        "placa": "ABC-123",
5        "marca": "Toyota",
6        "modelo": "Corolla"
7      },
8      {
9        "placa": "XYZ-456",
10       "marca": "Chevrolet",
11       "modelo": "Spark"
12     },
13     {
14       "placa": "MNO-789",
15       "marca": "Yamaha",
16       "modelo": "FZ25"
17     },
18     {
19       "placa": "PQR-321",
20       "marca": "Renault",
21       "modelo": "Logan"
22     },
23     {
24       "placa": "GHI-987",
25       "marca": "Ford",
26       "modelo": "Escape"
27     }
28   ]
29 }
```

7. Adjunto **link** de video donde se explica cómo se realizó el objeto json, se importó y analizó en Mongo db:

<https://drive.google.com/file/d/188fvqcdmqmJ1VE76LwHxsiAXak4Onf6o/view?usp=sharing>

8. Adjunto **link** de video donde se explica cómo se realizó el objeto json, de forma más completa y explico el array:

<https://drive.google.com/file/d/1L55kPPyLQxQfTb3Ta1yfUEmgAZDYQj9p/view?usp=sharing>

9. Conclusión

El desarrollo de este trabajo permitió comprender de manera práctica el funcionamiento de las bases de datos no relacionales, específicamente MongoDB, desde la conceptualización del modelo JSON hasta la ejecución de operaciones reales sobre una colección de datos.

A través de la implementación del sistema de parqueadero se pudo evidenciar que MongoDB ofrece una gran flexibilidad al momento de modelar información, ya que no exige un esquema rígido como las bases de datos relacionales tradicionales, lo que facilita la adaptación del modelo de datos según las necesidades del proyecto.

Las operaciones CRUD ejecutadas de la colección, inserción de cinco documentos, consultas, actualizaciones del primer y último registro, eliminación del tercer documento y búsqueda por número de placa demostraron que MongoDB proporciona comandos intuitivos y eficientes para la gestión completa de la información.

Finalmente, se concluye que el dominio de las bases de datos NoSQL es una competencia fundamental en el perfil de un desarrollador de software moderno, dado que gran parte de las aplicaciones actuales hacen uso de este tipo de tecnologías para manejar grandes volúmenes de datos de forma ágil y escalable.